Assignment 3 (AM5920): Kanak Agarwal (ID25M802)

Question 1

Consider the following 1D Poisson equation,

$$\frac{d^2u}{dx^2} = f(x) \tag{1}$$

with boundary conditions,

$$u(x=0) = 0, \quad u(x=L) = u_L \tag{2}$$

Consider a solution of the form,

$$u(x) = \sin(2\pi x) + a\sin(2\pi\omega x) + \text{Noise} \tag{3}$$

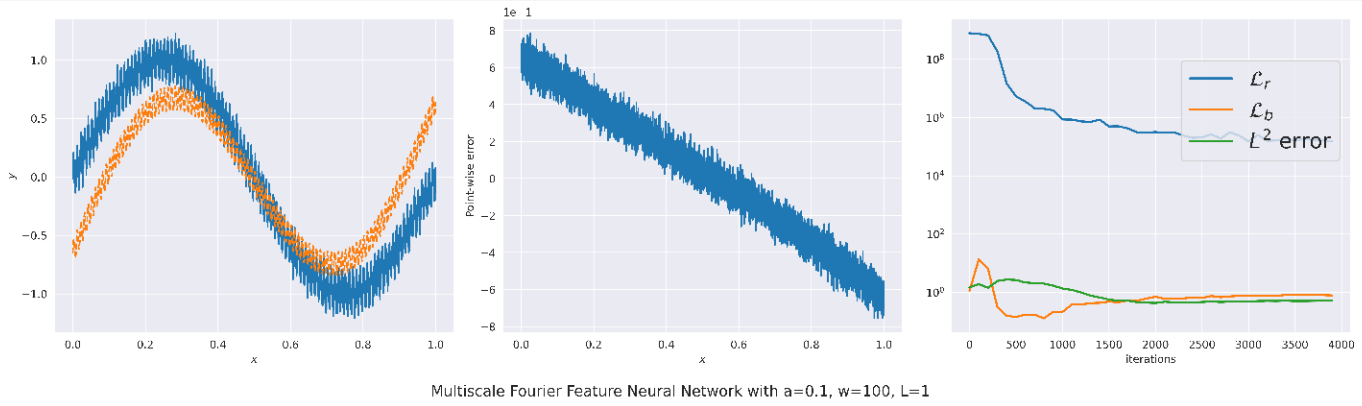
with,

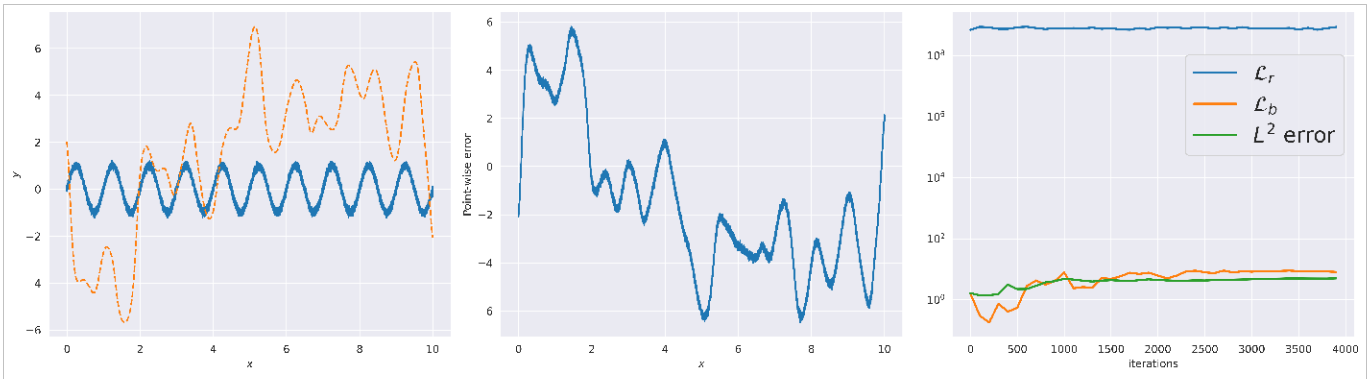
$$f(x) = -4\pi^2\sin(2\pi x) - 4\pi^2 a\omega^2\sin(2\pi\omega x) \tag{4}$$

eight cases are considered,

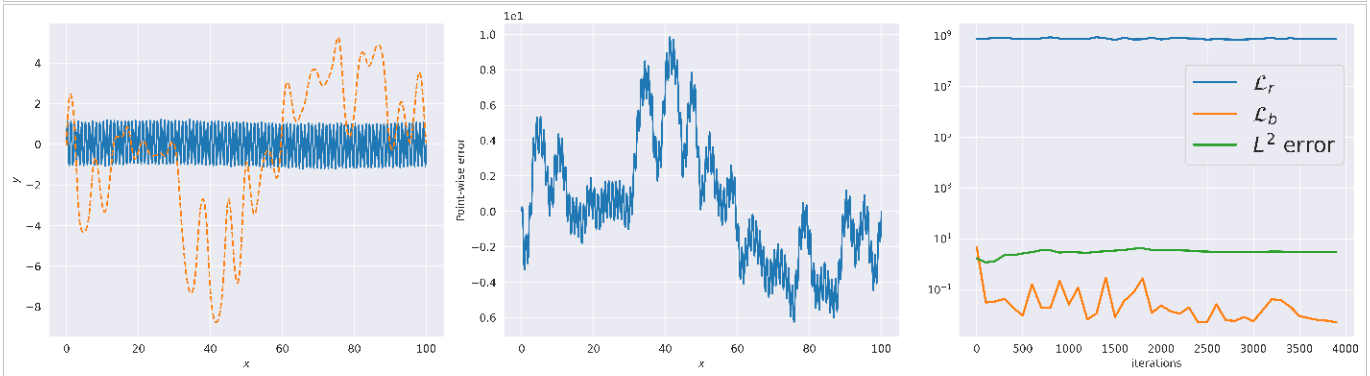
- Case 1: $a = 0.1, \omega = 100, L = 1$
- Case 2: $a = 10, \omega = 100, L = 1$
- Case 1: $a = 0.1, \omega = 100, L = 10$
- Case 2: $a = 10, \omega = 100, L = 10$
- Case 1: $a = 0.1, \omega = 100, L = 100$
- Case 2: $a = 10, \omega = 100, L = 100$
- Case 1: $a = 0.1, \omega = 100, L = 1000$
- Case 2: $a = 10, \omega = 100, L = 1000$

A multiscale fourier feature network with a tanh activation function is with 2 hidden layers with 100 neurons each. The method of *Wang et al.* (2021) is employed. The results are shown below,

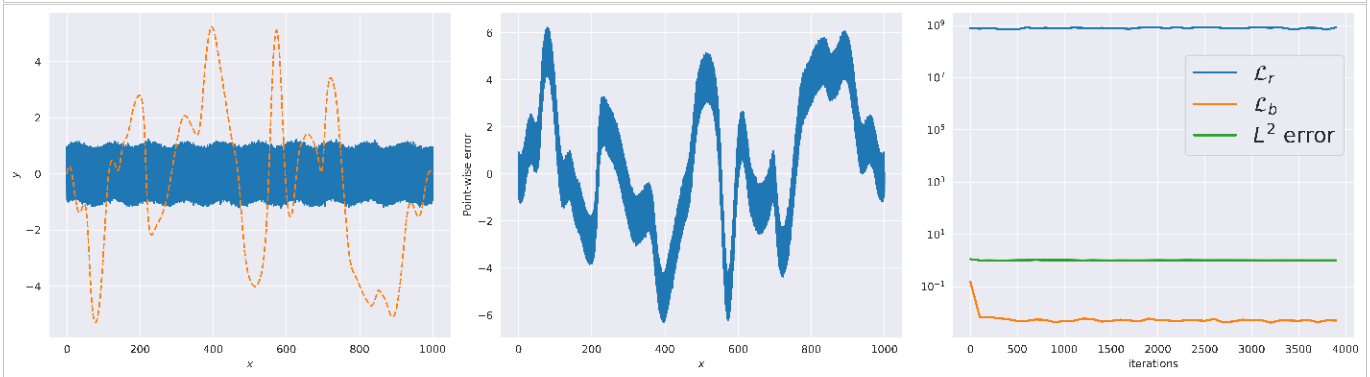




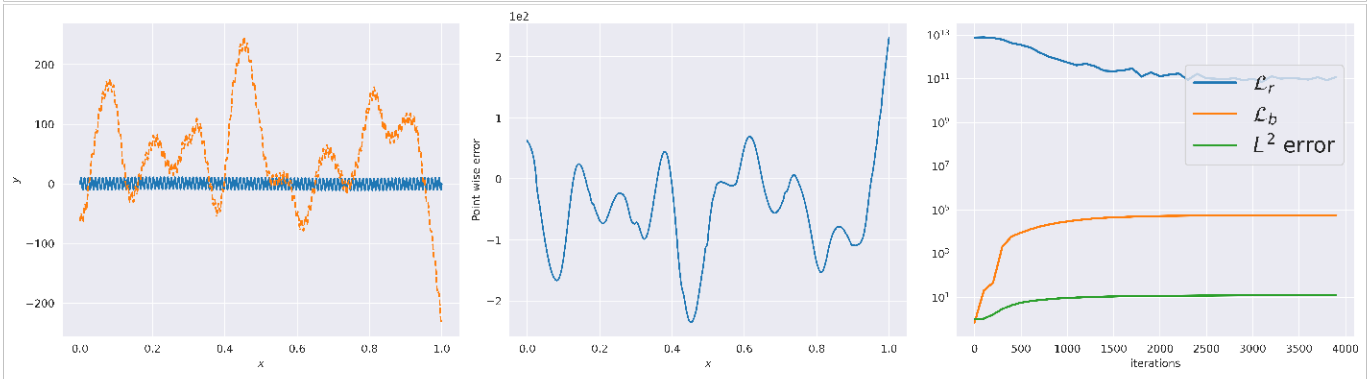
Multiscale Fourier Feature Neural Network with a=0.1, w=100, L=10



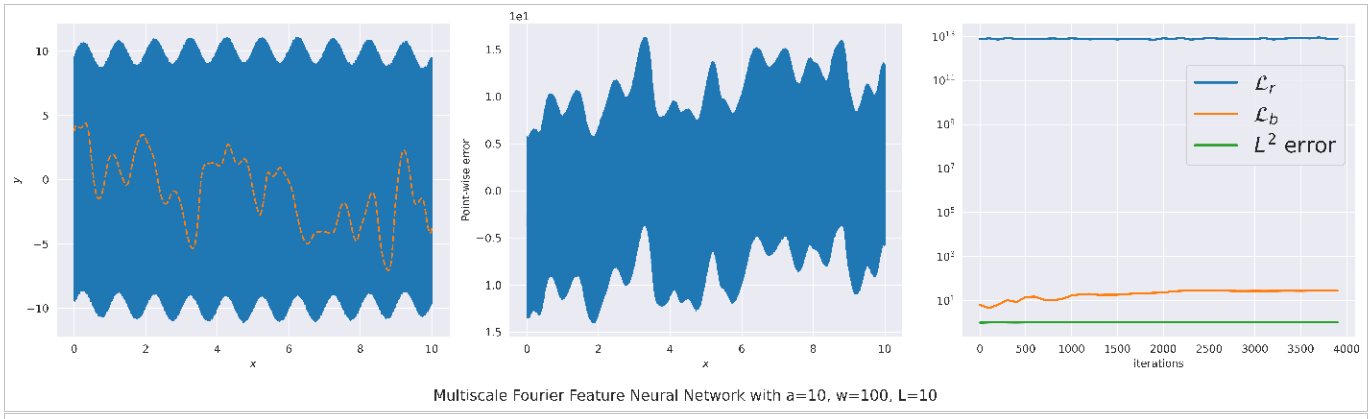
Multiscale Fourier Feature Neural Network with a=0.1, w=100, L=100



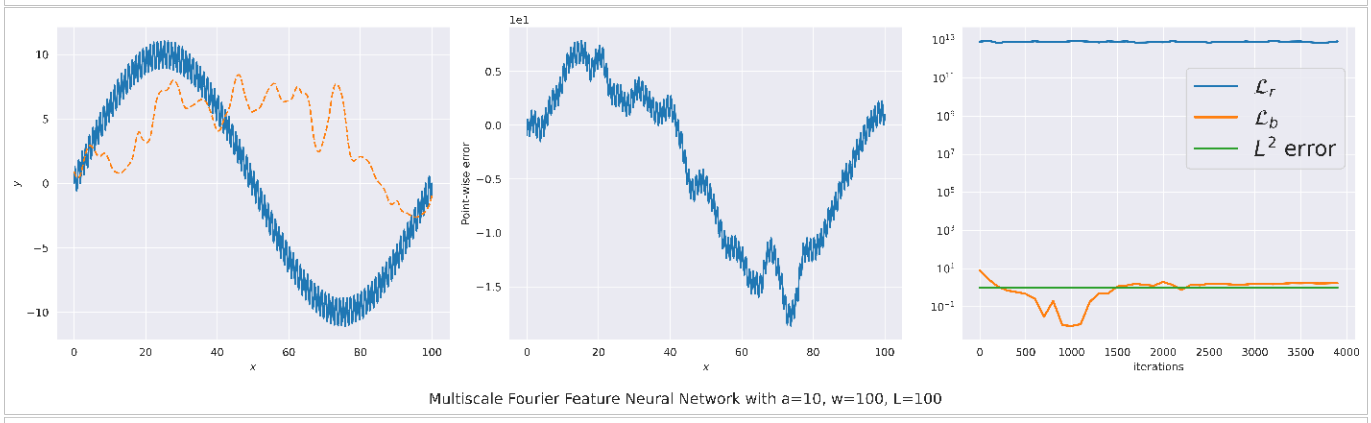
Multiscale Fourier Feature Neural Network with a=0.1, w=100, L=1000



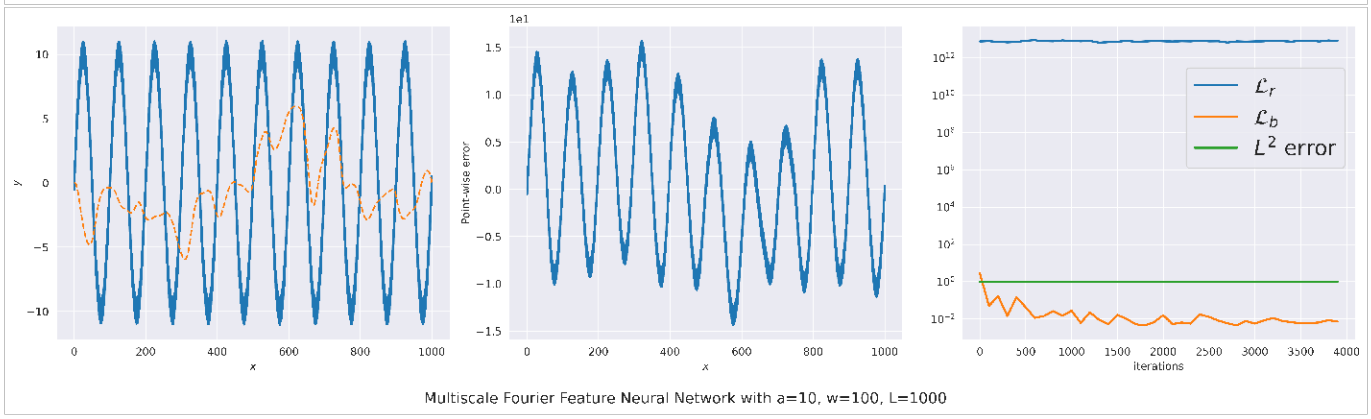
Multiscale Fourier Feature Neural Network with a=10, w=100, L=1



Multiscale Fourier Feature Neural Network with a=10, w=100, L=10



Multiscale Fourier Feature Neural Network with a=10, w=100, L=100



Multiscale Fourier Feature Neural Network with a=10, w=100, L=1000

The PINN gets worse with both an increase in a and L. The Conditionally Adaptive Augmented Lagrangian is also used on the same problem proposed by *Hu et al.* (2025). This is conducted using the same architecture with 5 trials. The results are shown below,

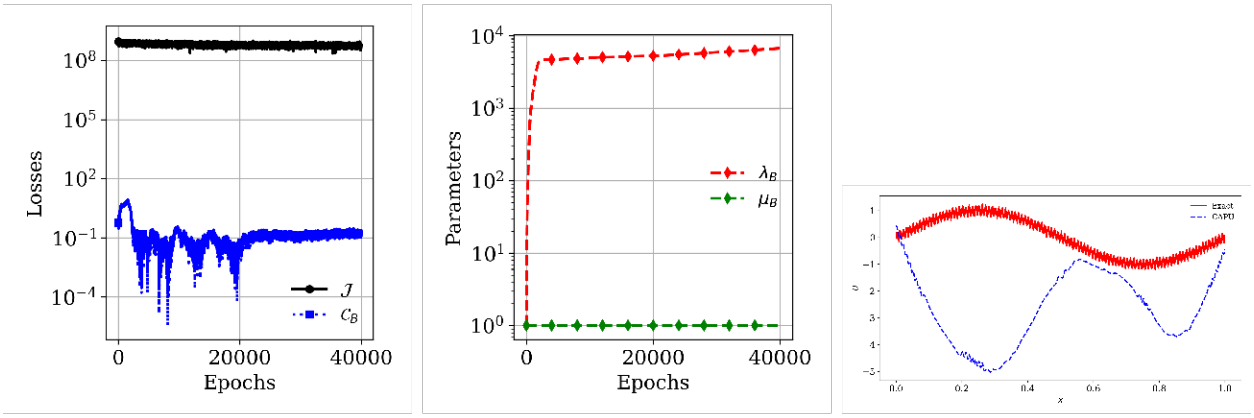


Figure 1: CAPU Results for $a = 0.1, \omega = 100, L = 1$

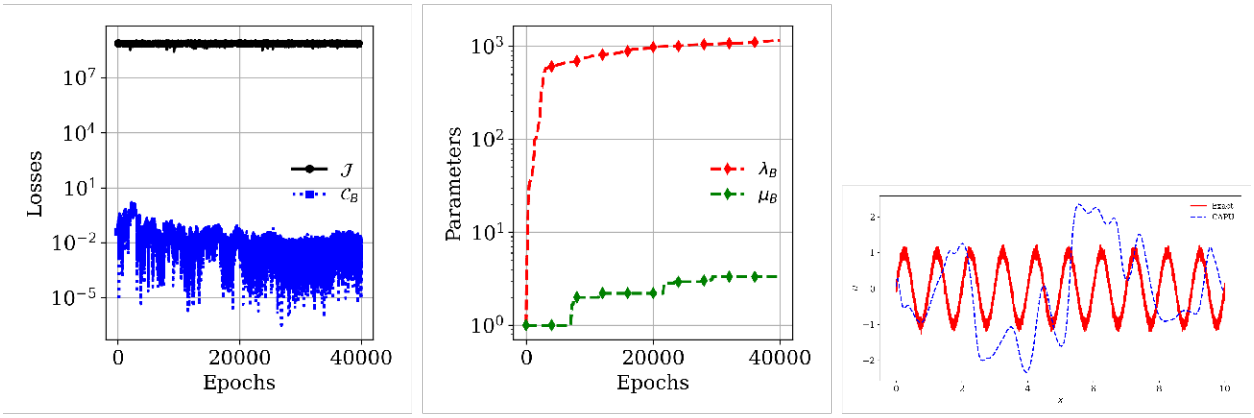


Figure 2: CAPU Results for $a = 0.1, \omega = 100, L = 10$

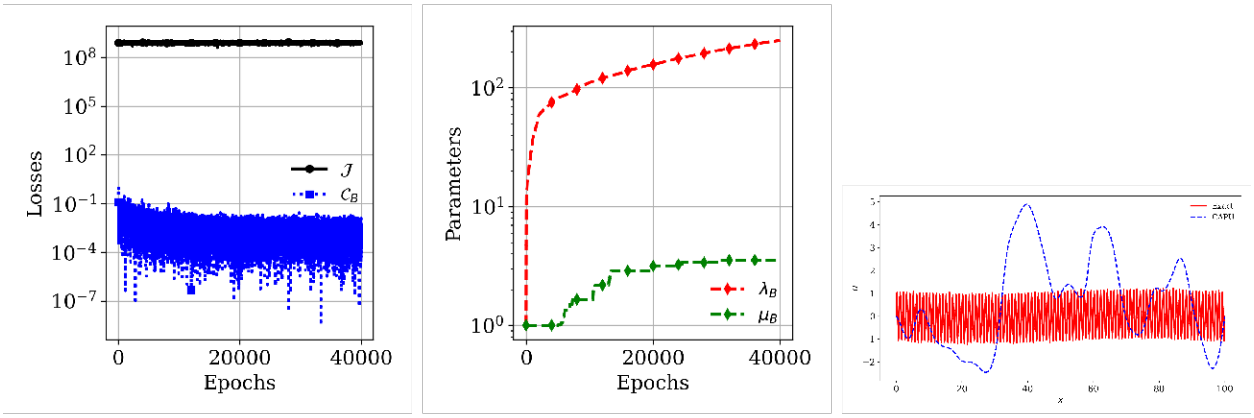


Figure 3: CAPU Results for $a = 0.1, \omega = 100, L = 100$

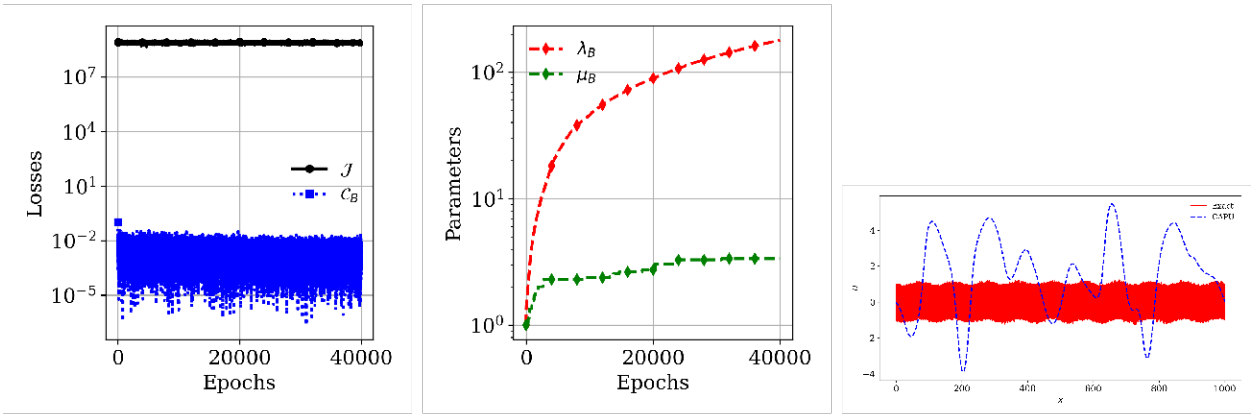


Figure 4: CAPU Results for $a = 0.1, \omega = 100, L = 1000$

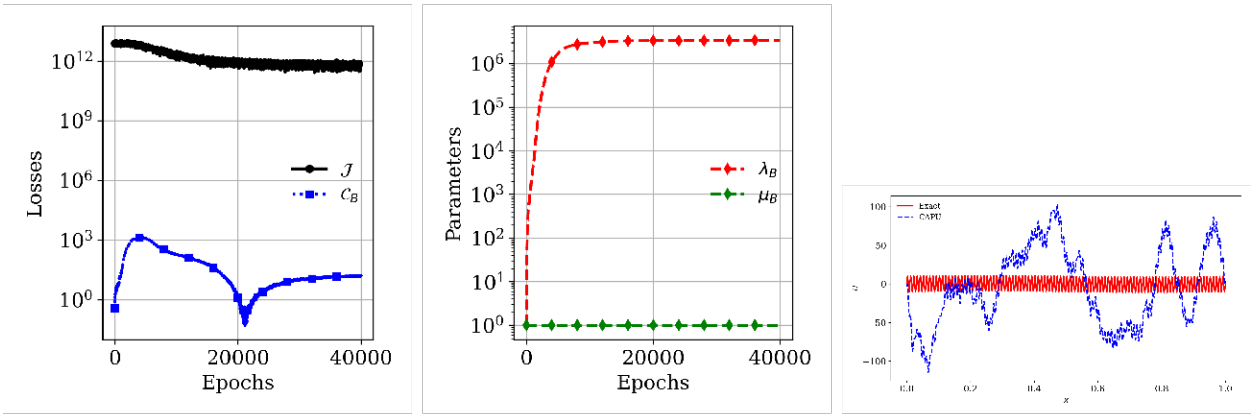


Figure 5: CAPU Results for $a = 10.0, \omega = 100, L = 1$

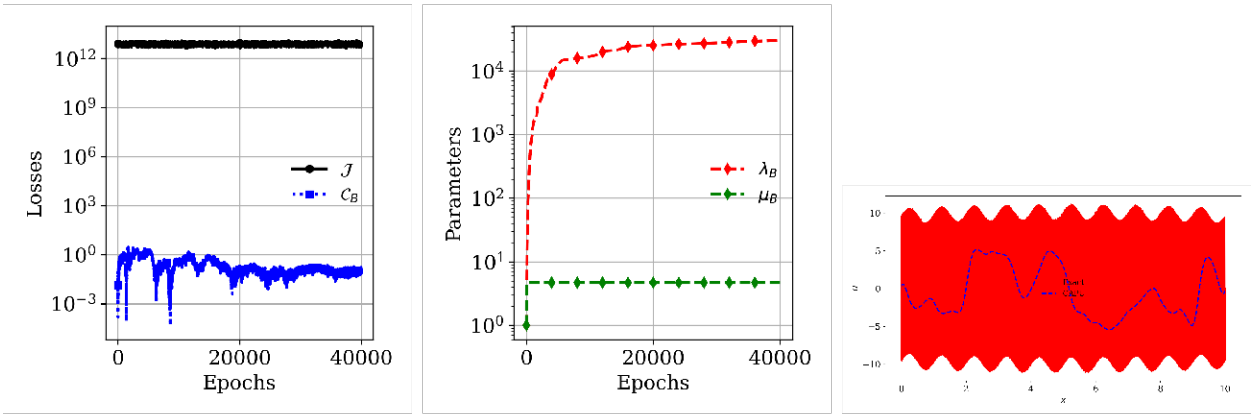


Figure 6: CAPU Results for $a = 10.0, \omega = 100, L = 10$

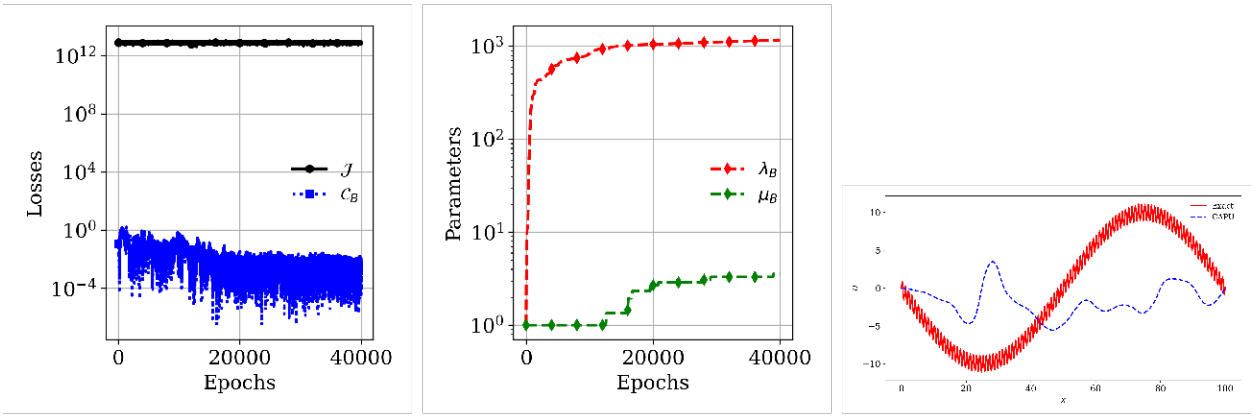


Figure 7: CAPU Results for $a = 10.0, \omega = 100, L = 100$

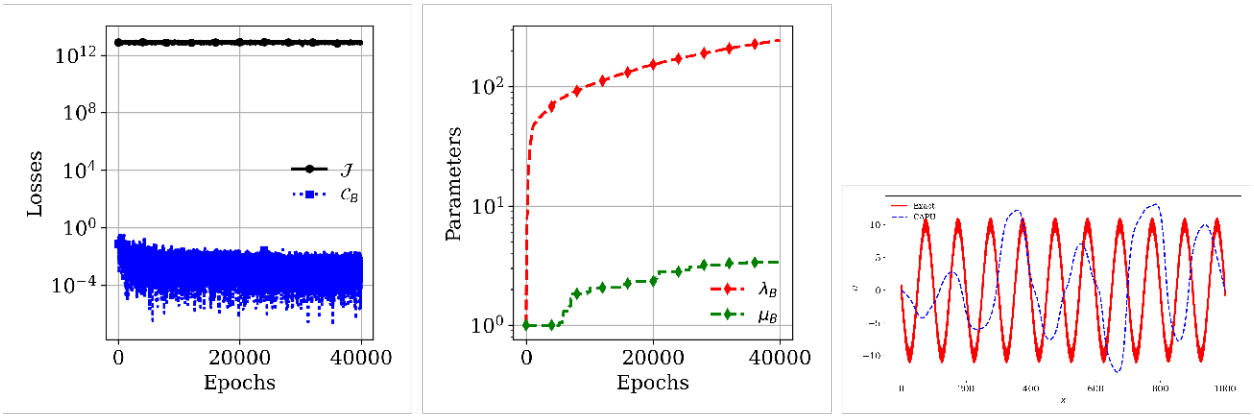


Figure 8: CAPU Results for $a = 10.0, \omega = 100, L = 1000$

Both convergence and accuracy depend on the size of the domain for both methods. Both CAPU and *Wang*'s method fail on the increase of a and L.

Question 2

Consider the following PDE,

$$\frac{d}{dx}\left(E(x)\frac{du}{dx}\right) = 0 \tag{5}$$

with boundary conditions,

$$u(x=0) = 0, \quad u(x=L) = u_L \tag{6}$$

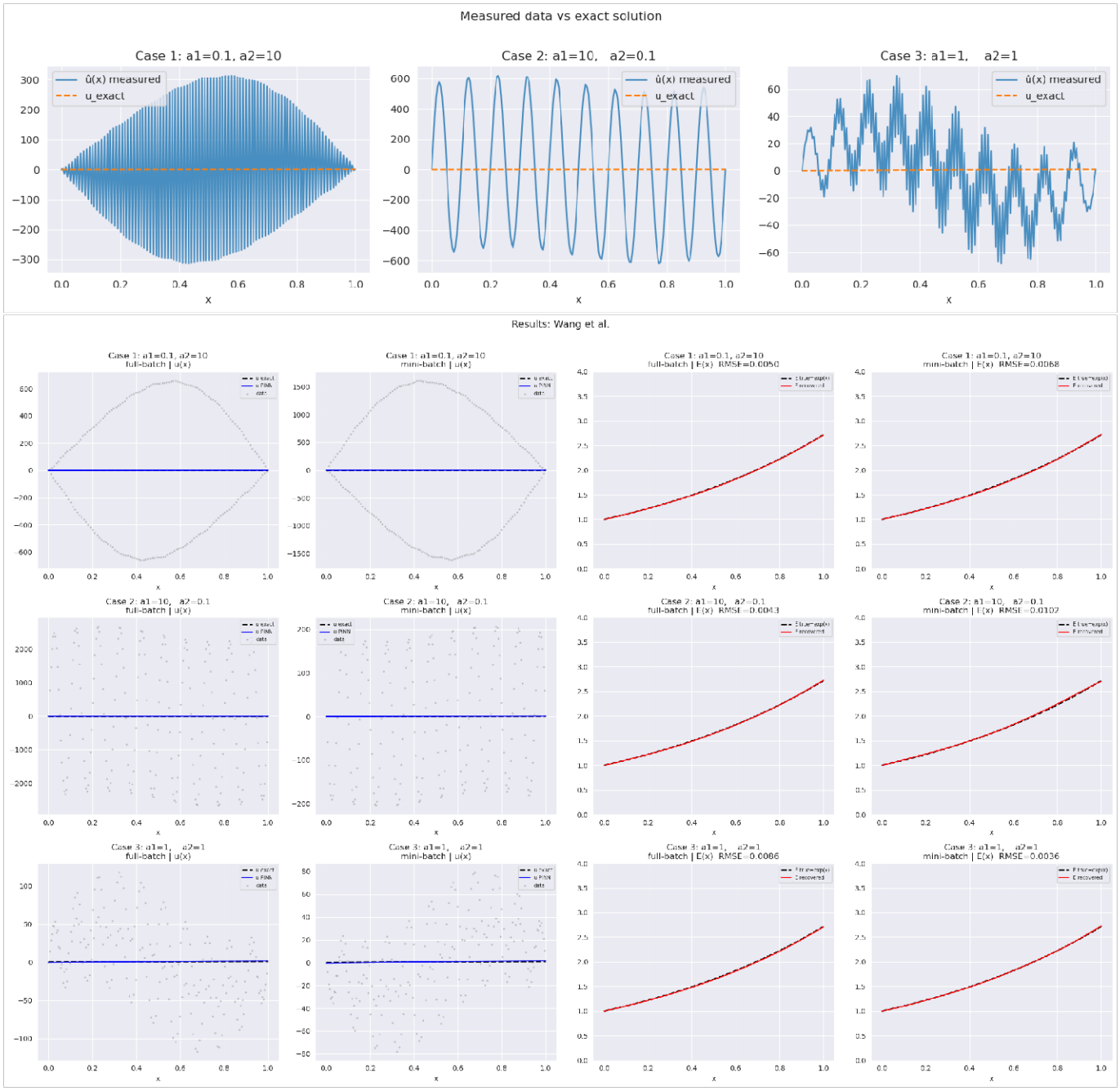
The experimental field is given by,

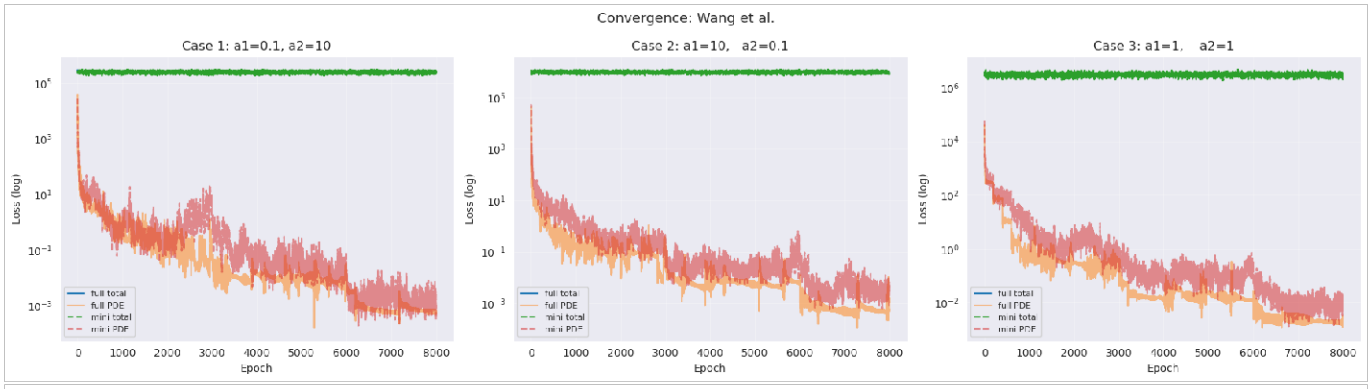
$$\hat{u}(x) = \sin(2\pi x) + a_1 \sin(20\pi x) + a_2 \sin(200\pi x) + \text{noise} \tag{7}$$

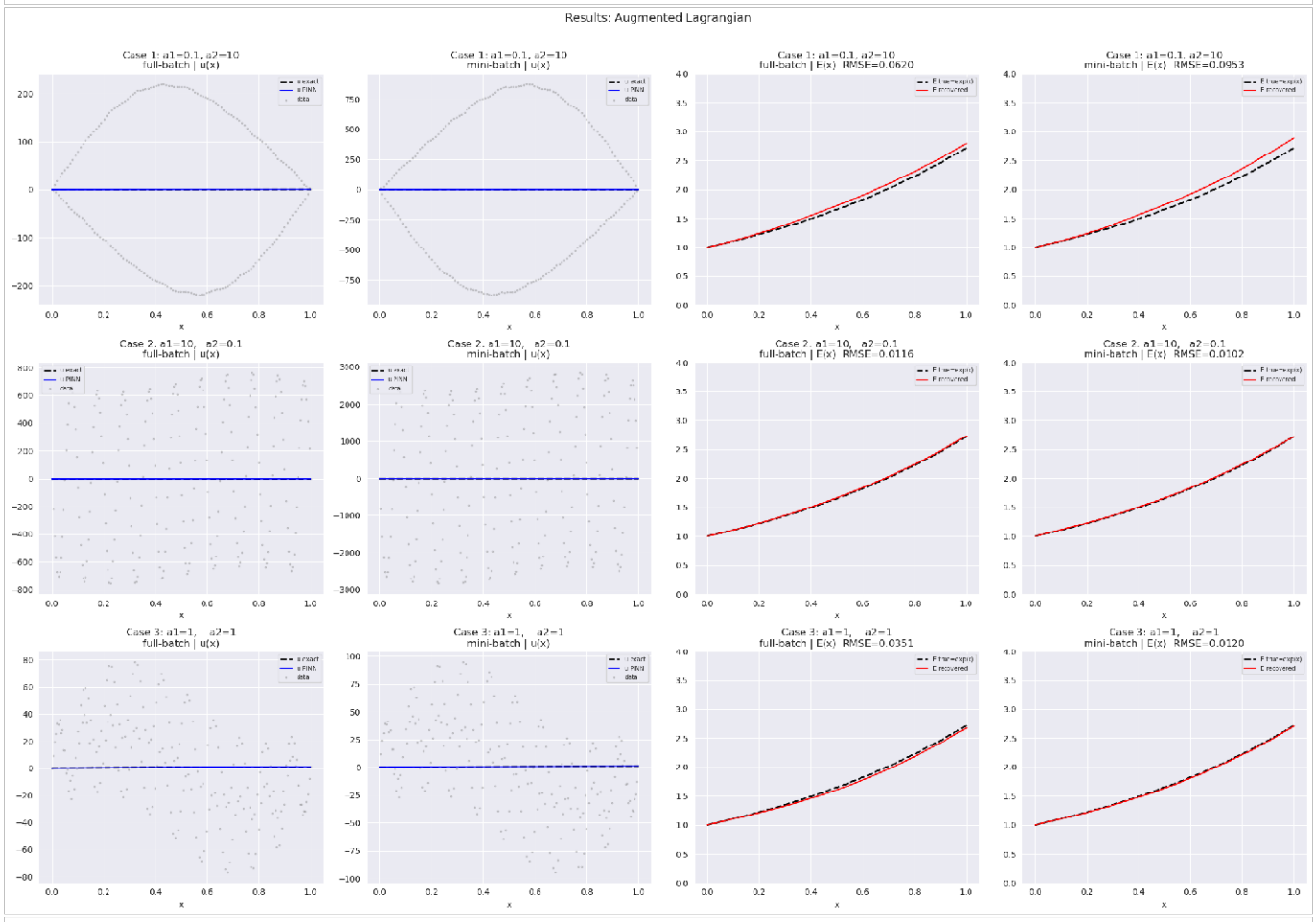
Three cases are considered:

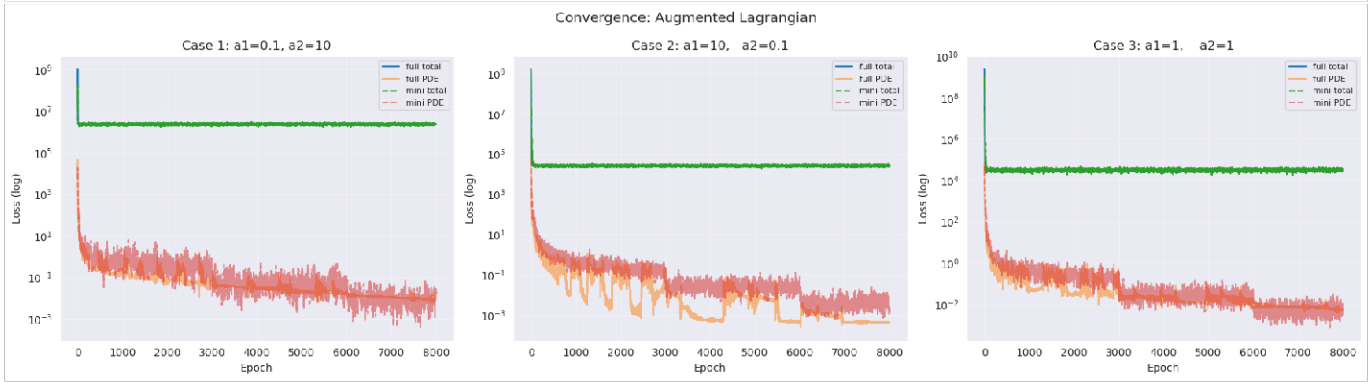
- $a_1 = 0.1, a_2 = 10$
- $a_1 = 10, a_2 = 0.1$
- $a_1 = 1, a_2 = 1$

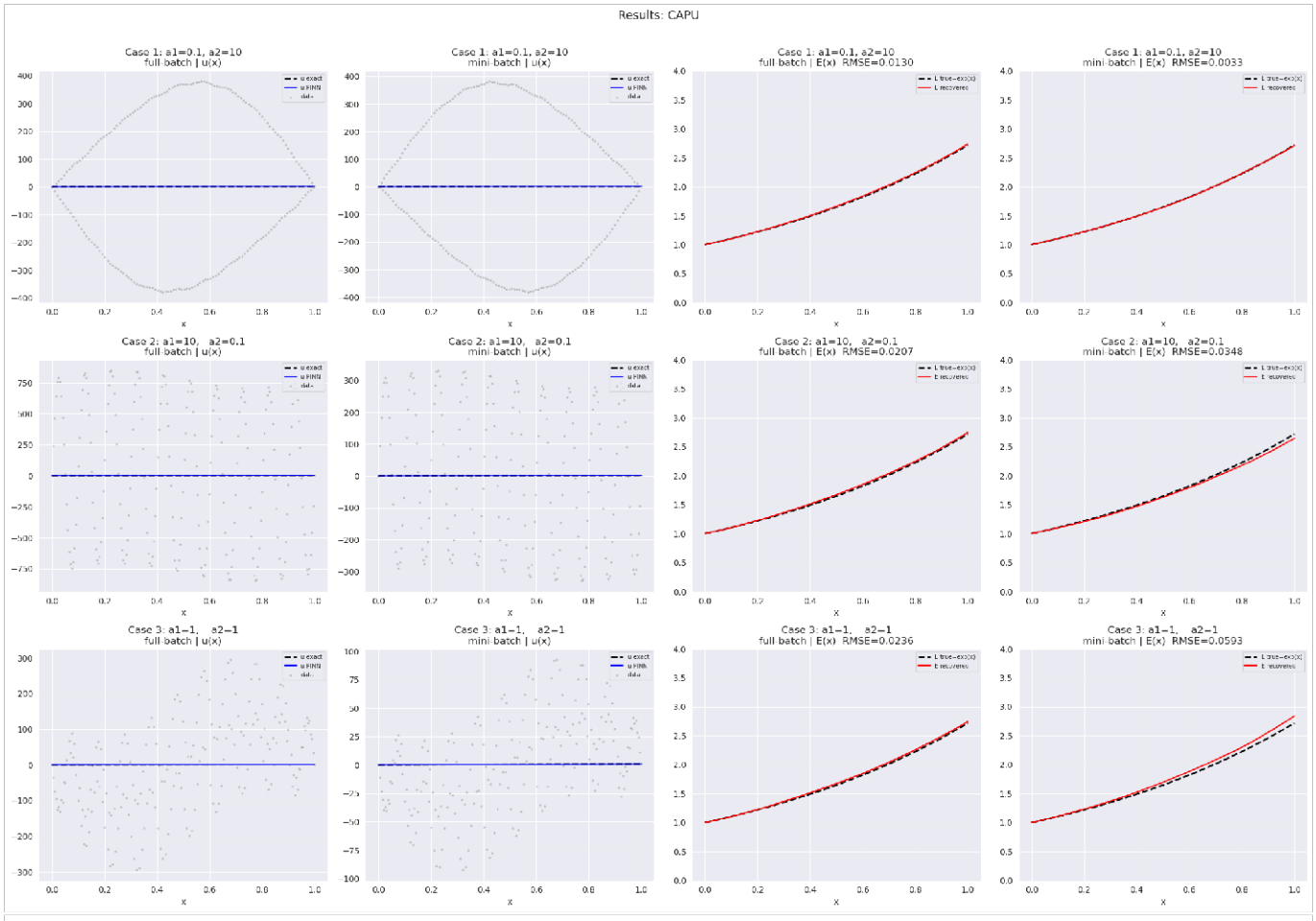
The results are as follows,

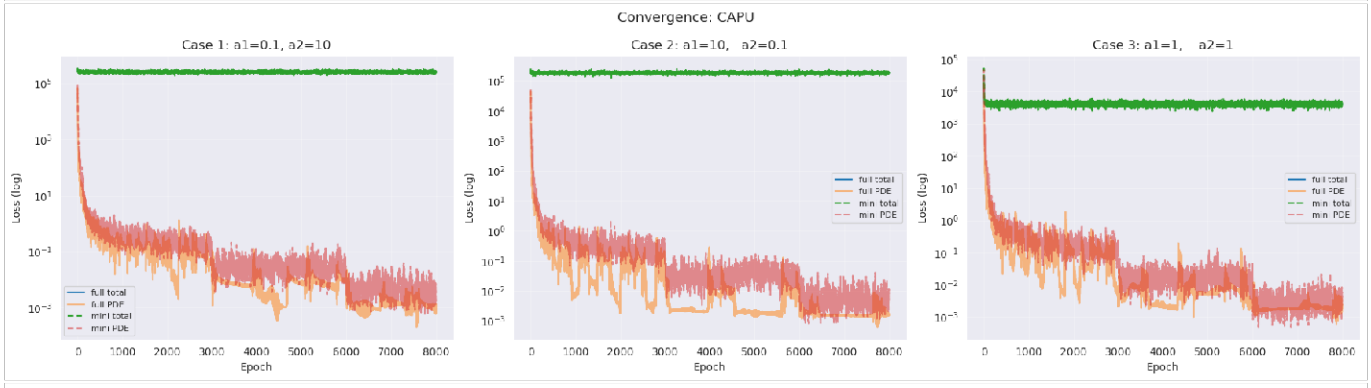


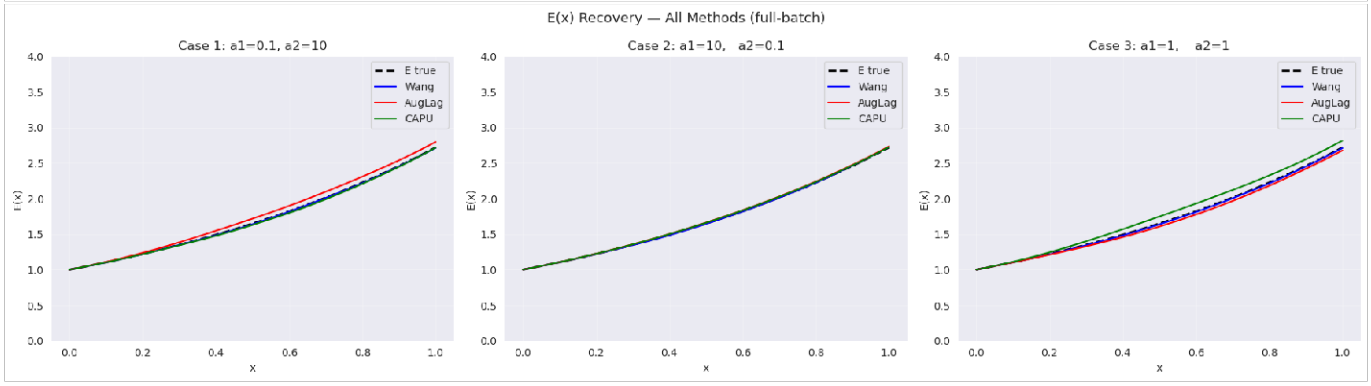

Convergence: Wang et al.
Case 1: a1=0.1, a2=10
Case 2: a1=10, a2=0.1
Case 3: a1=1, a2=1
Loss (log)
Epoch
full total
full PDE
mini total
mini PDE

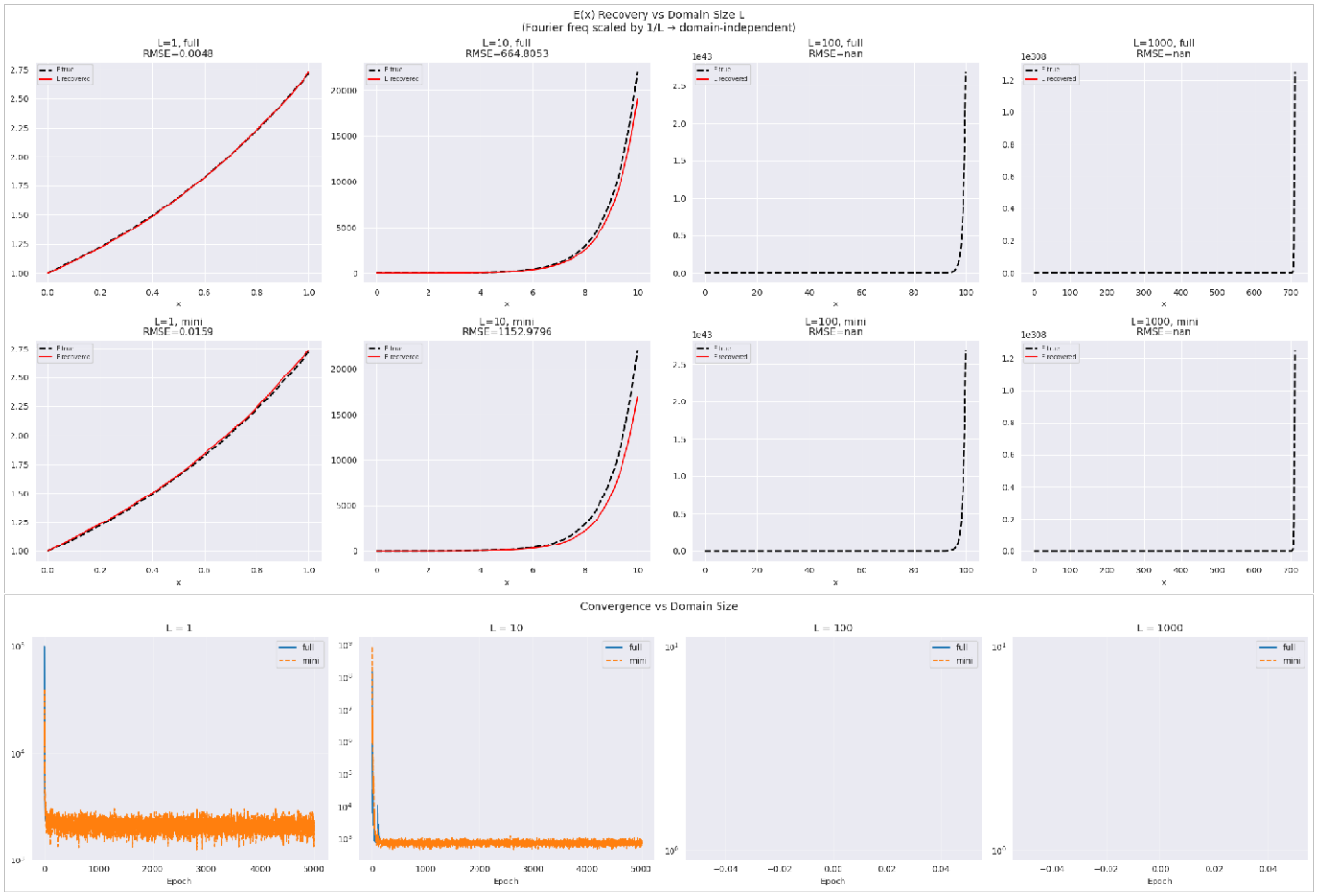

Results: Augmented Lagrangian
Case 1: a1=0.1, a2=10
full-batch | u(x)
mini-batch | u(x)
full-batch | E(x) RMSE=0.0620
mini-batch | E(x) RMSE=0.0953
Case 2: a1=10, a2=0.1
full-batch | u(x)
mini-batch | u(x)
full-batch | E(x) RMSE=0.0116
mini-batch | E(x) RMSE=0.0102
Case 3: a1=1, a2=1
full-batch | u(x)
mini-batch | u(x)
full-batch | E(x) RMSE=0.0351
mini-batch | E(x) RMSE=0.0120


Convergence: Augmented Lagrangian
Case 1: a1=0.1, a2=10
Case 2: a1=10, a2=0.1
Case 3: a1=1, a2=1
Loss (log)
Epoch
full total
full PDE
mini total
mini PDE


Results: CAPU
Case 1: a1=0.1, a2=10
full-batch | u(x)
Case 1: a1=0.1, a2=10
mini-batch | u(x)
Case 1: a1=0.1, a2=10
full-batch | E(x) RMSE=0.0130
Case 1: a1=0.1, a2=10
mini-batch | E(x) RMSE=0.0033
Case 2: a1=10, a2=0.1
full-batch | u(x)
Case 2: a1=10, a2=0.1
mini-batch | u(x)
Case 2: a1=10, a2=0.1
full-batch | E(x) RMSE=0.0207
Case 2: a1=10, a2=0.1
mini-batch | E(x) RMSE=0.0348
Case 3: a1=1, a2=1
full-batch | u(x)
Case 3: a1=1, a2=1
mini-batch | u(x)
Case 3: a1=1, a2=1
full-batch | E(x) RMSE=0.0236
Case 3: a1=1, a2=1
mini-batch | E(x) RMSE=0.0593


Convergence: CAPU
Case 1: a1=0.1, a2=10
Case 2: a1=10, a2=0.1
Case 3: a1=1, a2=1
Loss (log)
Epoch
full total
full PDE
mini total
mini PDE


E(x) Recovery — All Methods (full-batch)
Case 1: a1=0.1, a2=10
Case 2: a1=10, a2=0.1
Case 3: a1=1, a2=1
E true
Wang
AugLag
CAPU
E(x)
x



Wang's method of gradient balancing keeps data and PDE losses balanced, while the adaptive λ prevents either term from dominating. The full-batch training is smoother. The Augmented Lagrangian method enforces the PDE as a hard constraint via a growing multiplier, which is more robust than fixed weights. The multiplier grows until the PDE is satisfied. The Conditionally Adaptive Augmented Lagrangian only updates the multiplier when the PDE is still being violated, making it the most stable and consistently best in terms of RMSE. Scaling the Fourier frequency by 1/L is essential for both methods; without it, the network input range grows with L, causing training to fail. With scaling, the network always sees x mapped to a manageable range. The flux-constancy PDE loss also uses an exponential term that is domain-aware. Full-batch training is smoother and more accurate, while mini-batch training is faster but noisier and can miss the true solution for harder cases.